

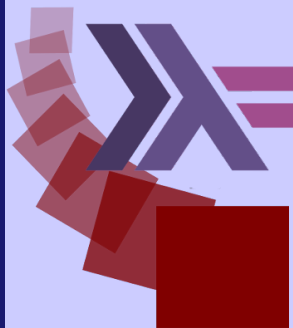


PROGRAMMATION FONCTIONNELLE

via le langage Haskell

OUALI Abdelkader

L2 au département Mathématique-Informatique
UFR des sciences
Université de Caen Normandie



Autres paradigmes : différents **styles** d'écriture de programme. un informaticien jongle avec ces styles selon les **besoins**.

Programmation fonctionnelle

- Application des fonctions aux arguments
- Principe de récurrence et composition de fonction
- Langages spécialisés : Haskell, Elixir, etc.

Présent dans la plupart de langages (C++, JAVA, Python, VisualBasic, C#, etc.)

- ▶ Alonzo Church (1930) :
 - ▶ λ -calcul
 - ▶ Théorie simple mais puissante de fonctions pour faire des calculs

- ▶ Une fonction est une **boîte noire** sans état interne :

$$\lambda x.x + 1 \text{ ou } \lambda x.\lambda y.x + y$$

- ▶ Variables
 - ▶ Construction de la fonction (notation λ)
 - ▶ Application par substitution des variables par les valeurs en entrée
- ▶ Les termes lambda peuvent émuler les machines de Turing et inversement.

Dans un terminal, lancer l'interpréteur GHCI.
Commandes de bases utiles dans GHCI :

- ▶ `> :?` – Help! Afficher toutes les commandes
- ▶ `> :load test` – Ouvrir le fichier test.hs
- ▶ `> :reload` – Recharge le fichier précédemment chargé
- ▶ `> :main a1 a2` – Invoque main avec les arguments de ligne de commande a1 a2
- ▶ `> :!` – Exécuter une commande shell
- ▶ `> :set editor nano` – configurer un éditeur qui sera utilisé à l'intérieur de GHCI
- ▶ `> :edit nom` – Modifier le script nom.hs
- ▶ `> :edit` – modifier le script actuel
- ▶ `> :type expr` – Afficher le type d'une expression
- ▶ `> :quit` – Quitter GHCI

Les commandes peuvent être **abrégées**. Par exemple, `:r` pour `:reload`
Au démarrage, les définitions du **"Standard Prelude"** sont chargées.

- ▶ Les noms de fonctions et de paramètres doivent commencer par une lettre minuscule, par exemple myFun1, arg_x, personName, etc.
 - ➡ par convention, un argument de type liste contient "s" un suffixe s dans son nom, p.ex. xs, ns, nss, etc.)

- ▶ Une fonction est définie par une égalité :

```
carré x = x * x  
plus x y = x + y
```

- ▶ Dès que la fonction est définie, on peut l'appliquer par appel de son nom et les arguments :
 - ➡ dans d'autres langage p.ex. C : carré(5); et plus(3,2);

```
> carré 5  
25  
> plus 3 2  
5
```

- ▶ La définition d'une fonction "=" peut-être écrite sur la même ligne ou plusieurs ligne sur la colonne suivante.

Mathématiques	Haskell
$f(x)$	<code>f x</code>
$f(x,y)$	<code>f x y</code>
$f(g(x))$	<code>f (g y)</code>
$f(x, g(y))$	<code>f x (g y)</code>
$f(x) g(y)$	<code>(f x) * (g y)</code>

- Syntaxe avec les indentations :

- Chaque bloc de code qui doit être indenté

⇒ "if", "case", "where", "let" ou "do"

- Toutes les définitions dans un bloc de code sont indentées sur la même colonne en utilisant des espaces si un saut de ligne apparaît.

```
quicksort [] = []
quicksort (x:xs) = quicksort ys ++ [x] ++ quicksort zs
  where
    ys = [a | a <- xs, a <= x]
    zs = [b | b <- xs, b > x]
```

let et where permettent de définir des variables/fonctions locales qui ne sont utilisées que dans la même fonction.

- ▶ `let ... in ...:`

```
f = let y = 1+2
      z = 4+6
      in y+z
```

- ▶ `where :`

```
f = y+z
  where y = 1+2
        z = 4+6
```

- ▶ `let ... in ...:` est une expression contrairement au `where` qui est réservé pour faire une construction syntaxique

if et case sont des expressions conditionnelles qui permettent de prendre des décisions en fonction des valeurs des expressions

- `if expression then expression else expression` : contrairement au langage impératif le **else est obligatoire**

```
plusPetit x y = if (x < y) then x
               else y
```

- `case of` :

```
troisPremiers x = case x of
  1 -> "A"
  2 -> "B"
  3 -> "C"
  _ -> "*" -- pattern-matching pour le choix par défaut (voir plus tard)
```

- Commentaires :

```
-- commentaire de fin de ligne (sur une seule ligne)
{- commentaire
en multiligne {- et imbricable -} -}
```

- les parenthèses servent uniquement à indiquer des priorités :

```
f x = 2 * (x - 3)
```

- et non pour évaluer une fonction :

```
Prelude> f 4
2
```

- et notamment quand un constructeur de valeur a plusieurs arguments :

```
Prelude> fmap (+ 3) (Just 5)
Just 8
```

- ▶ Entiers : `Int` et `Integer` avec les opérateurs (`==`, `/=`, `+`, `-`, `*`, `^`, et `div`)
- ▶ Flottant : `Float` et `Double` avec les opérateurs (`==`, `/=`, `+`, `-`, `*`, `^`, et `/`)
- ▶ Caractère : `Char` avec les opérateurs (`==`, `/=`, `succ`, `pred`)
- ▶ Liste : `[ , ]` avec les opérateurs (`==`, `/=`, `++`, `:`, `!!`)
- ▶ Tuple : `( , )` (si deux éléments `fst` et `snd`)
- ▶ Chaîne de caractères : `[Char]` ou `String`

- Un type rigide :

```
-- nomFonction :: TypePremierArgument -> TypeDeuxièmeArgument -> ... -> TypeRésultat
plusInt :: Int -> Int -> Int
plusInt x y = x + y
```

- Un type polymorphe :

```
-- nomFonction :: (instanciation de classe) => typeVariable1 -> typeVariable2 -> ... -> typeVariableResultat
plusInt :: (Num a) => a -> a -> a
plusInt x y = x + y
```

- Sélectionner le premier élément d'une liste :

```
Prelude>head [1,2,3,4,5]  
1
```

- Supprimer le premier élément d'une liste :

```
Prelude>tail [1,2,3,4,5]  
[2,3,4,5]
```

- Sélectionner tous les éléments d'une liste sauf le dernier.

```
Prelude>init [1, 2, 3]  
[1, 2]
```

- Sélectionner le dernier élément d'une liste.

```
Prelude>last [1, 2, 3]  
[3]
```

- Sélectionner les n premiers éléments d'une liste :

```
Prelude>take 3 [1,2,3,4,5]  
[1,2,3]
```

- Supprimer les n premiers éléments d'une liste :

```
Prelude> drop 3 [1,2,3,4,5]  
[4,5]
```

- Inverser une liste :

```
Prelude> reverse [1,2,3,4,5]  
[5,4,3,2,1]
```

- Calculer la longueur d'une liste :

```
Prelude> length [1,2,3,4,5]  
5
```

- Calculer la somme d'une liste :

```
Prelude> sum [1,2,3,4,5]  
15
```

- Calculer le produit d'une liste :

```
Prelude> product [1,2,3,4,5]  
120
```

```
1  -- avec une expression conditionnelle pour séparer les cas de base des cas récurtifs
2  fact :: Integer -> Integer
3  fact n = if (n == 0) then 1
4           else n * (fact (n - 1))
5  -- avec le filtrage par motifs (pattern matching)
6  factM :: Integer -> Integer
7  factM 0 = 1
8  factM n = n * (factM (n - 1))
9
10 -- Récurtivité sur les listes :
11 -- avec une expression conditionnelle et fonctions standards
12 somme :: [Int] -> Int
13 somme xs = if (length xs ) == 0 then 0
14           else head xs + (somme (tail xs))
15 -- avec le filtrage par motifs (pattern matching)
16 sommeM :: [Int] -> Int
17 sommeM [] = 0
18 sommeM (x:xs) = x + (sommeM xs)
19
```

- ▶ `_` attrape tout sans liaison
- ▶ `[]` est le pattern de la liste vide
- ▶ `(x:xs)` est le pattern des listes non vides
- ▶ `(x:xs)` est le pattern des listes ayant au moins un élément
- ▶ `(_:xs)` le premier n'a pas de liaison, le reste des éléments est lié `xs`
- ▶ `(x:y:xs)` est le pattern des listes ayant au moins deux éléments

ZF-expression :

```
-- syntaxe de base : ensemble => liste en Haskell
> [x | x <- [1..10]]

-- une valeur qui doit vérifiée un prédicat (condition)
> [x | x <- [1..10], even x]

-- expression sur la valeur récupérée
> [x * x | x <- [1..10], even x]

-- produit cartésien entre x et y
> [(x,y) | x <- [1..2], y <- [5..7]]
```

- ▶ La notation $\lambda x.f(x)$ désigne la fonction : $x \rightarrow f(x)$
 - ▶ successeur sur les entiers : $\lambda x.(x + 1)$
 - ▶ multiplier un nombre par 2 : $\lambda x.(2 * x)$
 - ▶ être un entier strictement positif : $\lambda x.(x > 0)$
- ▶ La notation Haskell

```
\x -> x + 1  
\x -> x * 2  
\x -> x > 0
```

```
> (\x -> x + 1) 2    ==> 3  
> (\x -> x * 2) (7+5) ==> 24  
> (\x -> x > 0) (head [1, 2, 7]) ==> True
```

- Les fonctions avec plusieurs arguments peuvent également être passés à une autre fonction

Une signature qui déclare une fonction comme un type de retour :

```
add :: Int -> (Int -> Int)
add x y = x + y
```

- En math, les fonction n'accepte qu'un seul argument, les fonctions avec plusieurs arguments on utilise la même techniques (fonctions qui retournent des fonctions...)
-

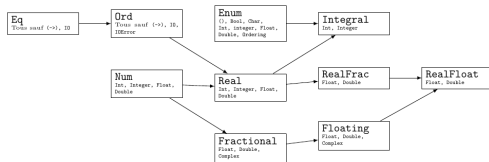
```
mult :: Int -> (Int -> (Int -> Int))
mult x y z = x * y * z
```

- Ce type de fonctions est appelé fonctions Curry

- ▶ Une **classe de types** définit un ensemble de fonctions (méthodes ou opérateur) partagées par plusieurs types
- ▶ Pour qu'un type appartienne à une classe de types, il doit implémenter les méthodes de cette classe de types

```
class Num a where
  (+) :: a -> a -> a
  (-) :: a -> a -> a
  (*) :: a -> a -> a
  negate :: a -> a
  abs :: a -> a
  signum :: a -> a
  fromInteger :: Integer -> a
```

classe	fonctions
Eq	== /=
Ord	< <= > >= min max
Num	+ - * negate abs signum
Enum	succ pred
Fractional	/ recip
Integral	div mod quot rem



- ▶ Encapsuler des modèles de programmation communs (schéma de programmation) en tant que fonctions grâce à la Curryfication.
- ▶ `(map f xs)` construit la liste des applications de la fonction `f` à tous les éléments de la liste `xs`

```
> map square [1..4]    ==> [1,4,9,16]
> map odd [1..4]       ==> [True, False, True, False]
> map (* 2) [1..4]     ==> [2, 4, 6, 8]
> map even [1..4]      ==> [False, True, False, True]
```

- ▶ `(filter p xs)` construit la liste des éléments de la liste `xs` satisfaisant le prédicat `p`

```
> filter even [1..10]   ==> [2,4,6,8,10]
> filter odd [1..10]    ==> [1,3,5,7,9]
> filter (== 'a') "azertayaap" ==> "aaaa"
> filter (/= 'a') "azertayaap" ==> "zertyp"
```

- `foldr` associe à droite i.e. l'application commence par la fin de liste

```
foldr op e [x1, x2, ..., xn] =  
    x1 op (x2 op (x3 ... op (xn op e) ...))
```

```
foldr (+) 0 [2,5,7,9] = 2 + (5 + (7 + (9 + 0))) = 23
```

```
foldr (*) 1 [2,5,7,9] = 2 * (5 * (7 * (9 * 1))) = 630
```

- `foldl` associe à gauche : on commence l'application par le début de liste

```
foldl op e [] = e
```

```
foldl op e [x1,...,xn] = (...((e op x1) op x2) ... op xn)
```

- Vérifier si tous les éléments d'une liste satisfont un prédicat :

```
> all even [2, 4, 6, 8]  
True
```

- Vérifier si un élément d'une liste satisfait un prédicat :

```
> any odd [2, 4, 6, 8]  
False
```

- Retourner les éléments dans une liste si le prédicat est satisfait :

```
Prelude> takeWhile (/=' ') "abc def"  
"abc"
```

- Retourner les éléments dans une liste en Enlevant les éléments satisfaisant le prédicat :

```
Main> dropWhile (/=' ') "abc def"  
" def"
```

- ▶ Le nom d'un nouveau type doit commencer par une majuscule.

```
type String = [Char ]
```

- ▶ Les déclarations de type peuvent être imbriquées.

```
type Board = [Pos ]  
type Pos = (Int , Int)
```

- ▶ Toutefois, les déclarations de type ne peuvent pas être récursives

Les nouvelles valeurs (constructeurs) sont utilisées comme les valeurs primitives :

- passés en **arguments** aux fonctions
- **renvoyés** en tant que résultats de fonctions
- **stockés** dans des **structures** de données telles que des listes ou autres

```
-- constructeur de valeurs sans paramètres
data Deplacement = Haut | Bas | Gauche | Droite deriving (Eq)

-- constructeur de valeurs avec paramètres
data Voyageur = Voyageur {nom::String, age::Int}

-- arbre non étiqueté
data Arbre a = Feuille a | Noeud (Arbre a) (Arbre a) deriving Show

-- arbre étiqueté
data Arbre2 a = Vide | NoeudEtiqu a (Arbre2 a) (Arbre2 a) deriving Show
```

- Fonction qui applique un mouvement à une position :

```
-- applique un déplacement à une position courante
deplace :: Deplacement -> Position -> Position
deplace Gauche (x , y) = (x - 1, y)
deplace Droite (x , y) = (x + 1, y)
deplace Haut (x , y) = (x , y - 1)
deplace Bas (x , y) = (x , y + 1)
```

- Une nouvelle classe peut être déclarée en utilisant le système **classe** d'Haskell
- La classe `MyEq` est déclarée comme suit :

```
class MyEq a where
  (==), (/=)  :: a -> a -> Bool
  -- une définition par défaut de la différence
  -- en fonction de l'éagilité qui sera prochainement définie
  x /= y = not (x == y)
```

- déclarer une instance de cette classe ne nécessite qu'une définition de `==` pour le type concerné :

```
instance MyEq Arbre where
  (Feuille x) == (Feuille y) = x == y
  (Noeud l1 x1 r1) == (Noeud l2 x2 r2) = (l1 == l2) && (x1 == x2) && (r1 == r2)
  _ == _ = False

-- interpréteur ghci
*Main> arbre = (Noeud (Noeud (Feuille 2) 3 (Feuille 4)) 5 (Noeud (Feuille 6) 7 (Feuille 9)))
*Main> arbre' = (Noeud (Noeud (Feuille 2) 3 (Feuille 4)) 5 (Feuille 6))
*Main> arbre == arbre'
False
*Main> arbre == arbre
True
*Main> arbre /= arbre
False
*Main> :t (==)
(==)  :: MyEq a => a -> a -> Bool
```